

Guía de Ejercicios K-Nearest Neighbors (KNN)

Analítica de Datos, Universidad de San Andrés

Si encuentran algún error en el documento o hay alguna duda, manden un mail a rodriguezf@udesa.edu.ar y lo revisamos.

1. Ejercicios

1.1. Ejercicios Conceptuales

1.1.1. Verdadero o Falso

1. KNN es un algoritmo de aprendizaje supervisado que requiere entrenamiento previo.
2. El parámetro k siempre debe ser impar para evitar empates en clasificación.
3. KNN es sensible a la escala de las variables de entrada.
4. Un valor de k muy grande puede llevar a overfitting.
5. La distancia euclidiana es la única métrica que se puede usar en KNN.
6. KNN puede manejar tanto clasificación como regresión.
7. El algoritmo KNN almacena todos los datos de entrenamiento en memoria.
8. Un valor de $k = 1$ siempre clasifica correctamente todos los puntos de entrenamiento.
9. KNN es un algoritmo paramétrico.
10. La normalización de variables es opcional en KNN.

1.1.2. Multiple Choice

1. ¿Cuál de las siguientes NO es una ventaja de KNN?
 - a) No requiere entrenamiento
 - b) Simple de entender

- c) Maneja datos faltantes automáticamente
 - d) No hace suposiciones sobre la distribución de datos
2. Para un problema de clasificación binaria con $k=3$, si los 3 vecinos más cercanos son $[1, 0, 1]$, la predicción será:
- a) 0
 - b) 1
 - c) No se puede determinar
 - d) Error del algoritmo
3. ¿Qué sucede si k es igual al número total de puntos de entrenamiento?
- a) El algoritmo falla
 - b) Siempre predice la clase mayoritaria
 - c) No es posible
 - d) Predice la clase del punto más cercano
4. ¿Cuál es el efecto de aumentar k en un problema de clasificación?
- a) Mayor overfitting
 - b) Mayor underfitting
 - c) No tiene efecto
 - d) Depende del dataset
5. ¿Qué métrica de distancia es más apropiada para variables categóricas?
- a) Euclidiana
 - b) Manhattan
 - c) Hamming
 - d) Minkowski

1.2. Ejercicios de Clasificación

1.2.1. Clasificación Simple

Tenemos los siguientes datos de entrenamiento para clasificar flores:

Longitud Pétalo	Ancho Pétalo	Clase
1.4	0.2	Setosa
1.4	0.2	Setosa
1.3	0.2	Setosa
4.7	1.4	Versicolor
4.5	1.5	Versicolor
4.9	1.5	Versicolor
6.0	2.5	Virginica
5.1	1.9	Virginica
5.9	2.1	Virginica

Un nuevo punto tiene longitud de pétalo = 4.8 y ancho de pétalo = 1.6.

1. Calculá la distancia euclidiana a todos los puntos de entrenamiento.
2. Encontrá los 3 vecinos más cercanos ($k=3$).
3. ¿Cuál es la clasificación del nuevo punto?

1.2.2. Efecto del Parámetro k

Usando los mismos datos del ejercicio anterior:

1. Clasificá el punto (4.8, 1.6) usando $k=1$.
2. Clasificá el mismo punto usando $k=5$.
3. Clasificá el mismo punto usando $k=7$.
4. ¿Por qué pueden diferir los resultados?

1.2.3. Normalización

Tenemos datos de estudiantes con las siguientes características:

Edad	Altura (cm)	Clase
20	165	Aprobado
22	170	Aprobado
19	160	Aprobado
25	180	Desaprobado
24	175	Desaprobado
26	182	Desaprobado

1. Calculá las distancias al punto (21, 168) sin normalizar.
- b) Normalizá los datos usando min-max scaling.
- c) Calculá las distancias normalizadas.
- d) ¿Cómo cambian los resultados?

1.2.4. Múltiples Variables

Tenemos datos de clientes bancarios:

Edad	Ingresos	Deuda	Clase
25	30000	5000	Riesgo Bajo
30	45000	8000	Riesgo Bajo
35	40000	12000	Riesgo Bajo
45	35000	25000	Riesgo Alto
50	75000	30000	Riesgo Alto
55	55000	35000	Riesgo Alto

1. Calculá las distancias al punto (38, 70000, 15000) sin normalizar.
2. Encontrá los 3 vecinos más cercanos al punto sin normalizar y clasificá el nuevo cliente.
3. Normalizá los datos.

4. Encontrá los 3 vecinos más cercanos al punto normalizado y clasificá el nuevo cliente.
5. ¿Cómo cambian los resultados con y sin normalización?

1.2.5. Datos Desbalanceados

Tenemos un dataset con 100 puntos de clase A y 10 puntos de clase B:

1. Si $k=5$, ¿qué problema puede surgir?
2. ¿Cómo se puede solucionar este problema?
3. ¿Qué valor de k sería más apropiado?

1.2.6. Curse of Dimensionality

1. Explicá qué es la curse of dimensionality.
2. ¿Por qué afecta especialmente a KNN?
3. ¿Qué técnicas se pueden usar para mitigarla?

1.2.7. Métricas de Distancia

Tenemos dos puntos: A(1, 2, 3) y B(4, 6, 9).

1. Calculá la distancia euclidiana.
2. Calculá la distancia Manhattan.
3. Calculá la distancia de Minkowski con $p=3$.
4. ¿Cuál es la relación entre estas distancias?

1.2.8. Pesos por Distancia

En lugar de votación simple, usamos pesos inversos a la distancia:

1. Si las distancias son [2, 4, 6] y las clases [A, B, A], calculá la predicción con pesos.
2. Compará con la votación simple.
3. ¿Cuándo es útil usar pesos?

1.2.9. Implementación

Implementá KNN desde cero en Python:

1. Escribí la función de distancia euclidiana.
2. Escribí la función para encontrar k vecinos más cercanos.
3. Escribí la función de clasificación.
4. Probá con los datos del ejercicio 1.

1.3. Ejercicios de Regresión

1.3.1. Regresión Simple

Tenemos datos de precios de casas:

Área (m ²)	Precio (USD)
50	80000
60	95000
70	110000
80	125000
90	140000
100	155000

1. Predicé el precio de una casa de 75 m² usando k=3.
2. ¿Cuál es la diferencia con clasificación?

1.3.2. Múltiples Variables

Tenemos datos de ventas:

Publicidad	Precio	Ventas
1000	10	500
1500	12	600
2000	15	700
2500	18	800
3000	20	900

1. Normalizá los datos usando min-max scaling.
2. Calculá las distancias del punto (1800, 14) a todos los puntos normalizados.
3. Encontrá los 3 vecinos más cercanos.
4. Predicé las ventas para publicidad=1800, precio=14 usando $k=3$.
5. ¿Cómo cambiaría la predicción si usaras $k=5$?

1.3.3. Efecto de k en Regresión

1. ¿Qué pasa si $k=1$ en regresión?
2. ¿Qué pasa si $k=n$ (número total de puntos)?
3. ¿Cuál es el valor óptimo de k ?

1.3.4. Pesos en Regresión

1. Explicá cómo funcionan los pesos en regresión KNN.
2. ¿Cuándo es mejor usar pesos que promedio simple?
3. Calculá la predicción ponderada para el ejercicio 11.

1.3.5. Outliers

Tenemos datos de precios de casas con un outlier:

Área (m ²)	Precio (USD)
50	80000
60	95000
70	110000
80	125000
90	140000
100	155000
95	500000

1. Predicé el precio de una casa de 85 m² usando $k=3$ con y sin el outlier.

2. ¿Cómo afectan los outliers a KNN en regresión?
3. ¿Qué técnicas se pueden usar para mitigar su efecto?
4. Compará con otros algoritmos de regresión.

1.3.6. Selección de Variables

Tenemos 10 variables predictoras:

1. ¿Cómo elegirías las variables más importantes?
2. ¿Qué técnicas de selección de características funcionan bien con KNN?
3. ¿Por qué es importante la selección de variables en KNN?

1.3.7. Comparación con Otros Métodos

1. Compará KNN con regresión lineal.
2. Compará KNN con regresión polinomial.
3. ¿Cuándo preferirías cada uno?

1.3.8. Análisis de Datos Manual

Tenemos datos de estudiantes con sus notas en 3 materias y si aprobaron o no:

Matemática	Física	Química	Aprobó
85	78	82	Sí
72	65	70	No
90	88	85	Sí
68	72	75	No
88	85	80	Sí
75	70	68	No
92	90	88	Sí
70	68	72	No
85	80	78	Sí
65	62	68	No

1. Normalizá los datos usando min-max scaling.
2. Calculá las distancias euclidianas del punto (80, 75, 78) a todos los puntos.
3. Encontrá los 3 vecinos más cercanos.
4. ¿Cuál es la predicción para este estudiante?
5. ¿Cómo cambiaría la predicción si usaras $k=5$?

2. Soluciones

2.1. Ejercicios Conceptuales

2.1.1. Verdadero o Falso

1. **Falso.** KNN no requiere entrenamiento previo, es un algoritmo lazy.
2. **Falso.** k puede ser par, pero impar evita empates en clasificación binaria.
3. **Verdadero.** KNN es muy sensible a la escala de las variables.
4. **Falso.** Un k muy grande lleva a underfitting, no overfitting.
5. **Falso.** Se pueden usar múltiples métricas de distancia.
6. **Verdadero.** KNN funciona para ambos tipos de problemas.
7. **Verdadero.** KNN almacena todos los datos de entrenamiento.
8. **Verdadero.** Con $k=1$, cada punto se clasifica como su vecino más cercano.
9. **Falso.** KNN es no paramétrico.
10. **Falso.** La normalización es crucial en KNN.

2.1.2. Multiple Choice

1. **c)** KNN no maneja datos faltantes automáticamente.
2. **b)** 1 (mayoría de votos).
3. **b)** Siempre predice la clase mayoritaria.
4. **d)** Depende del dataset específico.
5. **c)** Hamming es apropiada para variables categóricas.

2.2. Ejercicios de Clasificación

2.2.1. Clasificación Simple

1. Distancias euclidianas a todos los puntos:

- (1.4, 0.2) Setosa: $\sqrt{(4,8 - 1,4)^2 + (1,6 - 0,2)^2} = \sqrt{11,56 + 1,96} = 3,68$
- (1.4, 0.2) Setosa: $\sqrt{(4,8 - 1,4)^2 + (1,6 - 0,2)^2} = \sqrt{11,56 + 1,96} = 3,68$
- (1.3, 0.2) Setosa: $\sqrt{(4,8 - 1,3)^2 + (1,6 - 0,2)^2} = \sqrt{12,25 + 1,96} = 3,77$
- (4.7, 1.4) Versicolor: $\sqrt{(4,8 - 4,7)^2 + (1,6 - 1,4)^2} = \sqrt{0,01 + 0,04} = 0,22$
- (4.5, 1.5) Versicolor: $\sqrt{(4,8 - 4,5)^2 + (1,6 - 1,5)^2} = \sqrt{0,09 + 0,01} = 0,32$
- (4.9, 1.5) Versicolor: $\sqrt{(4,8 - 4,9)^2 + (1,6 - 1,5)^2} = \sqrt{0,01 + 0,01} = 0,14$
- (6.0, 2.5) Virginica: $\sqrt{(4,8 - 6,0)^2 + (1,6 - 2,5)^2} = \sqrt{1,44 + 0,81} = 1,50$
- (5.1, 1.9) Virginica: $\sqrt{(4,8 - 5,1)^2 + (1,6 - 1,9)^2} = \sqrt{0,09 + 0,09} = 0,42$
- (5.9, 2.1) Virginica: $\sqrt{(4,8 - 5,9)^2 + (1,6 - 2,1)^2} = \sqrt{1,21 + 0,25} = 1,21$

2. Los 3 vecinos más cercanos son:

- a) (4.9, 1.5) - Versicolor (distancia: 0.14)
- b) (4.7, 1.4) - Versicolor (distancia: 0.22)
- c) (5.1, 1.9) - Virginica (distancia: 0.42)

3. Clasificación: Versicolor (2 votos vs 1 voto de Virginica)

2.2.2. Efecto del Parámetro k

1. Con k=1: Versicolor (punto más cercano)

2. Con k=5: Los 5 vecinos más cercanos son: (4.9,1.5) Versicolor, (4.7,1.4) Versicolor, (5.1,1.9) Virginica, (4.5,1.5) Versicolor, (5.9,2.1) Virginica. Resultado: Versicolor (3 votos vs 2)

3. Con k=7: Los 7 vecinos más cercanos incluyen: 4 Versicolor, 2 Virginica, 1 Setosa. Resultado: Versicolor (4 votos vs 2 vs 1)

4. Los resultados pueden diferir porque con $k=1$ solo se considera el vecino más cercano, mientras que con $k=5$ o $k=7$ se consideran más puntos, lo que puede cambiar la clasificación si hay puntos de otras clases cercanos. En este caso, Versicolor mantiene la mayoría en todos los escenarios.

2.2.3. Normalización

1. Distancias sin normalizar:

- A (20, 165): $\sqrt{(21 - 20)^2 + (168 - 165)^2} = \sqrt{1 + 9} = 3,16$
- B (22, 170): $\sqrt{(21 - 22)^2 + (168 - 170)^2} = \sqrt{1 + 4} = 2,24$
- C (19, 160): $\sqrt{(21 - 19)^2 + (168 - 160)^2} = \sqrt{4 + 64} = 8,25$
- D (25, 180): $\sqrt{(21 - 25)^2 + (168 - 180)^2} = \sqrt{16 + 144} = 12,65$
- E (24, 175): $\sqrt{(21 - 24)^2 + (168 - 175)^2} = \sqrt{9 + 49} = 7,62$
- F (26, 182): $\sqrt{(21 - 26)^2 + (168 - 182)^2} = \sqrt{25 + 196} = 14,87$

2. Datos normalizados (min-max scaling):

Edad Norm	Altura Norm	Clase
0.14	0.23	Aprobado
0.43	0.45	Aprobado
0.00	0.00	Aprobado
0.86	0.91	Desaprobado
0.71	0.68	Desaprobado
1.00	1.00	Desaprobado

3. Distancias normalizadas:

- A: $\sqrt{(0,29 - 0,14)^2 + (0,36 - 0,23)^2} = 0,20$
- B: $\sqrt{(0,29 - 0,43)^2 + (0,36 - 0,45)^2} = 0,15$
- C: $\sqrt{(0,29 - 0,00)^2 + (0,36 - 0,00)^2} = 0,46$
- D: $\sqrt{(0,29 - 0,86)^2 + (0,36 - 0,91)^2} = 0,78$

- E: $\sqrt{(0,29 - 0,71)^2 + (0,36 - 0,68)^2} = 0,50$

- F: $\sqrt{(0,29 - 1,00)^2 + (0,36 - 1,00)^2} = 0,95$

4. Los resultados cambian significativamente. Sin normalización, la variable altura domina por su mayor escala, mientras que con normalización ambas variables tienen igual peso.

2.2.4. Múltiples Variables

1. Distancias SIN Normalizar (k=3)

El punto a clasificar es **P(38, 70000, 15000)**. Calculamos la distancia euclidiana a todos los puntos de entrenamiento:

$$\begin{aligned} A(25, 30000, 5000) &= \sqrt{(38 - 25)^2 + (70000 - 30000)^2 + (15000 - 5000)^2} \\ &= \sqrt{13^2 + 40000^2 + 10000^2} \\ &= 41231 \end{aligned}$$

$$\begin{aligned} B(30, 45000, 8000) &= \sqrt{(38 - 30)^2 + (70000 - 45000)^2 + (15000 - 8000)^2} \\ &= \sqrt{8^2 + 25000^2 + 7000^2} \\ &= 25962 \end{aligned}$$

$$\begin{aligned} C(35, 40000, 12000) &= \sqrt{(38 - 35)^2 + (70000 - 40000)^2 + (15000 - 12000)^2} \\ &= \sqrt{3^2 + 30000^2 + 3000^2} \\ &= 30150 \end{aligned}$$

$$\begin{aligned} D(45, 35000, 25000) &= \sqrt{(38 - 45)^2 + (70000 - 35000)^2 + (15000 - 25000)^2} \\ &= \sqrt{(-7)^2 + 35000^2 + (-10000)^2} \\ &= 36401 \end{aligned}$$

$$\begin{aligned}
E(50, 75000, 30000) &= \sqrt{(38 - 50)^2 + (70000 - 75000)^2 + (15000 - 30000)^2} \\
&= \sqrt{(-12)^2 + (-5000)^2 + (-15000)^2} \\
&= 15811
\end{aligned}$$

$$\begin{aligned}
F(55, 55000, 35000) &= \sqrt{(38 - 55)^2 + (70000 - 55000)^2 + (15000 - 35000)^2} \\
&= \sqrt{(-17)^2 + 15000^2 + (-20000)^2} \\
&= 25000
\end{aligned}$$

2. **Vecinos:** E (Riesgo Alto), F (Riesgo Alto), B (Riesgo Bajo).

Clasificación: Riesgo Alto (2 votos vs 1).

3. **Normalizando**

Punto P(38, 70000, 15000) se normaliza a **P_norm(0.43, 0.89, 0.33)**.

Punto	Edad Norm	Ingresos Norm	Deuda Norm	Clase
A	0.00	0.00	0.00	RB
B	0.17	0.33	0.10	RB
C	0.33	0.22	0.23	RB
D	0.67	0.11	0.67	RA
E	0.83	1.00	0.83	RA
F	1.00	0.56	1.00	RA

4. **Distancias Normalizadas**

Ahora, calculamos las distancias desde **P_norm(0.43, 0.89, 0.33)**:

$$\begin{aligned}
A_norm &= \sqrt{(0,43 - 0)^2 + (0,89 - 0)^2 + (0,33 - 0)^2} \approx 1,04 \\
B_norm &= \sqrt{(0,43 - 0,17)^2 + (0,89 - 0,33)^2 + (0,33 - 0,1)^2} \approx \mathbf{0.66} \\
C_norm &= \sqrt{(0,43 - 0,33)^2 + (0,89 - 0,22)^2 + (0,33 - 0,23)^2} \approx \mathbf{0.68} \\
D_norm &= \sqrt{(0,43 - 0,67)^2 + (0,89 - 0,11)^2 + (0,33 - 0,67)^2} \approx 0,88 \\
E_norm &= \sqrt{(0,43 - 0,83)^2 + (0,89 - 1)^2 + (0,33 - 0,83)^2} \approx \mathbf{0.65} \\
F_norm &= \sqrt{(0,43 - 1)^2 + (0,89 - 0,56)^2 + (0,33 - 1)^2} \approx 0,94
\end{aligned}$$

Vecinos: E_norm (Riesgo Alto), B_norm (Riesgo Bajo), C_norm (Riesgo Bajo).

Clasificación: Riesgo Bajo (2 votos vs 1).

5. La normalización cambió todo. Sin normalizar clasificaba como Riesgo Alto porque la variable ingresos dominaba.

2.2.5. Datos Desbalanceados

1. Con k=5, es muy probable que todos los vecinos sean de clase A, ignorando completamente la clase B.
2. Se puede solucionar usando:
 - Pesos por distancia
 - k más pequeño
 - Técnicas de sampling
 - Métricas de distancia específicas
3. Un k más pequeño (1-3) sería más apropiado para capturar la clase minoritaria.

2.2.6. Curse of Dimensionality

1. La curse of dimensionality es el fenómeno donde el rendimiento de algoritmos basados en distancia se degrada en espacios de alta dimensionalidad porque la distancia entre puntos se vuelve menos significativa.
2. Afecta a KNN porque:

- Las distancias se vuelven menos discriminativas
- El volumen del espacio crece exponencialmente
- Los puntos se vuelven equidistantes

3. Técnicas para mitigarla:

- Selección de características
- Reducción de dimensionalidad (PCA)
- Normalización apropiada
- Métricas de distancia específicas

2.2.7. Métricas de Distancia

1. Distancia euclidiana: $\sqrt{(4-1)^2 + (6-2)^2 + (9-3)^2} = \sqrt{9+16+36} = \sqrt{61} = 7,81$
2. Distancia Manhattan: $|4-1| + |6-2| + |9-3| = 3+4+6 = 13$
3. Distancia Minkowski (p=3): $\sqrt[3]{|4-1|^3 + |6-2|^3 + |9-3|^3} = \sqrt[3]{27+64+216} = \sqrt[3]{307} = 6,75$
4. La distancia de Minkowski generaliza las demas: p=1 es Manhattan, p=2 es Euclidiana, p=∞ es Chebyshev.

2.2.8. Pesos por Distancia

1. Pesos: $w_i = \frac{1}{d_i}$
 - Peso A: $1/2 = 0,5$
 - Peso B: $1/4 = 0,25$
 - Peso A: $1/6 = 0,167$

Voto ponderado: $(0,5 + 0,167)/(0,5 + 0,25 + 0,167) = 0,667$ para clase A

2. Con votación simple: A (2 votos vs 1 voto)
3. Los pesos son útiles cuando:
 - Los datos están desbalanceados
 - Se quiere dar más importancia a vecinos más cercanos
 - Se quiere suavizar la predicción

2.2.9. Implementación

```
1 import numpy as np
2
3 def euclidean_distance(x1, x2):
4     return np.sqrt(np.sum((x1 - x2) ** 2))
5
6 def get_neighbors(X_train, y_train, x_test, k):
7     distances = []
8     for i in range(len(X_train)):
9         dist = euclidean_distance(X_train[i], x_test)
10        distances.append((dist, y_train[i]))
11    distances.sort(key=lambda x: x[0])
12    return distances[:k]
13
14 def predict_classification(X_train, y_train, x_test, k):
15     neighbors = get_neighbors(X_train, y_train, x_test, k)
16     neighbor_classes = [neighbor[1] for neighbor in neighbors]
17     class_counts = {}
18     for clase in neighbor_classes:
19         if clase in class_counts:
20             class_counts[clase] += 1
21         else:
22             class_counts[clase] = 1
23     max_count = -1
24     prediccion = None
25     for clase in class_counts:
26         if class_counts[clase] > max_count:
27             max_count = class_counts[clase]
28             prediccion = clase
29     return prediccion
30
31 def predict_regression(X_train, y_train, x_test, k):
32     neighbors = get_neighbors(X_train, y_train, x_test, k)
33     neighbor_values = [neighbor[1] for neighbor in neighbors]
34     return sum(neighbor_values) / len(neighbor_values)
35
36 def predict_regression_weighted(X_train, y_train, x_test, k):
37     neighbors = get_neighbors(X_train, y_train, x_test, k)
38     weighted_sum = 0
39     weight_sum = 0
40     for distance, value in neighbors:
41         if distance == 0: # Evitar division por cero
42             return value
43         weight = 1 / distance
44         weighted_sum += weight * value
```

```

45         weight_sum += weight
46     return weighted_sum / weight_sum
47
48 # Datos del ejercicio 1
49 X_train = np.array([
50     [1.4, 0.2], [1.4, 0.2], [1.3, 0.2],
51     [4.7, 1.4], [4.5, 1.5], [4.9, 1.5],
52     [6.0, 2.5], [5.1, 1.9], [5.9, 2.1]
53 ])
54 y_train = ['Setosa', 'Setosa', 'Setosa', 'Versicolor', 'Versicolor',
55            'Versicolor', 'Virginica', 'Virginica', 'Virginica']
56
57 # Prediccion de clasificacion
58 x_test = np.array([4.8, 1.6])
59 prediction = predict_classification(X_train, y_train, x_test, 3)
60 print(f"Prediccion clasificacion: {prediction}")
61
62 # Ejemplo de regresion
63 X_reg = np.array([[50], [60], [70], [80], [90], [100]])
64 y_reg = np.array([80000, 95000, 110000, 125000, 140000, 155000])
65
66 x_test_reg = np.array([75])
67 prediction_reg = predict_regression(X_reg, y_reg, x_test_reg, 3)
68 print(f"Prediccion regresion: {prediction_reg}")
69
70 # Ejemplo de regresion con pesos
71 prediction_weighted = predict_regression_weighted(X_reg, y_reg,
72            x_test_reg, 3)
73 print(f"Prediccion regresion ponderada: {prediction_weighted}")

```

2.3. Ejercicios de Regresión

2.3.1. Regresión Simple

1. Para una casa de 75 m², los 3 vecinos más cercanos son:

- 70 m² → 110000 USD (distancia: 5)
- 80 m² → 125000 USD (distancia: 5)
- 60 m² → 95000 USD (distancia: 15)

Predicción: $(110000 + 125000 + 95000)/3 = 110000$ USD

2. En clasificación se vota por la clase mayoritaria, en regresión se promedia el valor objetivo.

2.3.2. Múltiples Variables

1. Normalización de datos (min-max scaling):

Rangos: Publicidad (2000), Precio (10)

Publicidad Norm	Precio Norm	Ventas
0.00	0.00	500
0.25	0.20	600
0.50	0.50	700
0.75	0.80	800
1.00	1.00	900

Punto a predecir: (1800, 14) se normaliza a (0.40, 0.40)

2. Distancias desde el punto normalizado (0.40, 0.40):

$$d_1 = \sqrt{(0,40 - 0,00)^2 + (0,40 - 0,00)^2} = \sqrt{0,16 + 0,16} = 0,57$$

$$d_2 = \sqrt{(0,40 - 0,25)^2 + (0,40 - 0,20)^2} = \sqrt{0,0225 + 0,04} = 0,25$$

$$d_3 = \sqrt{(0,40 - 0,50)^2 + (0,40 - 0,50)^2} = \sqrt{0,01 + 0,01} = 0,14$$

$$d_4 = \sqrt{(0,40 - 0,75)^2 + (0,40 - 0,80)^2} = \sqrt{0,1225 + 0,16} = 0,53$$

$$d_5 = \sqrt{(0,40 - 1,00)^2 + (0,40 - 1,00)^2} = \sqrt{0,36 + 0,36} = 0,85$$

3. Los 3 vecinos más cercanos son:

a) Punto 3: (2000, 15) → Ventas: 700 (distancia: 0.14)

b) Punto 2: (1500, 12) → Ventas: 600 (distancia: 0.25)

c) Punto 4: (2500, 18) → Ventas: 800 (distancia: 0.53)

4. Predicción con k=3: $(700 + 600 + 800)/3 = 700$ ventas

5. Con k=5: Se incluyen todos los puntos, predicción: $(500 + 600 + 700 + 800 + 900)/5 = 700$ ventas

En este caso particular, ambas predicciones dan el mismo resultado porque los datos están bien distribuidos.

2.3.3. Efecto de k en Regresión

1. Con $k=1$: La predicción es exactamente el valor del vecino más cercano.
2. Con $k=n$: La predicción es el promedio de todos los valores de entrenamiento.
3. El valor óptimo de k depende del dataset, pero generalmente está entre 3 y 10.

2.3.4. Pesos en Regresión

1. Los pesos en regresión KNN funcionan así:
 - $w_i = \frac{1}{d_i}$ donde d_i es la distancia
 - Predicción = $\frac{\sum w_i \cdot y_i}{\sum w_i}$
2. Los pesos son mejores cuando:
 - Los datos tienen mucha variabilidad local
 - Se quiere suavizar la predicción
 - Los vecinos cercanos son más confiables
3. Para el ejercicio 11:
 - Pesos: $1/5 = 0,2$, $1/5 = 0,2$, $1/15 = 0,067$
 - Predicción ponderada: $(0,2 \cdot 110000 + 0,2 \cdot 125000 + 0,067 \cdot 95000) / (0,2 + 0,2 + 0,067) = 113000$ USD

2.3.5. Outliers

1. Para una casa de 85 m²:
 - **Sin outlier:** Los 3 vecinos más cercanos son 80m² (125000), 90m² (140000), 70m² (110000). Predicción: $(125000 + 140000 + 110000) / 3 = 125000$ USD
 - **Con outlier:** Los 3 vecinos más cercanos son 80m² (125000), 90m² (140000), 95m² (500000). Predicción: $(125000 + 140000 + 500000) / 3 = 255000$ USD

El outlier infla la predicción en más del 100 %.

2. Los outliers afectan a KNN en regresión porque:

- Pueden ser los vecinos más cercanos
- Sesgan el promedio de predicción
- Afectan más que en otros algoritmos

3. Técnicas para mitigar:

- Usar mediana en lugar de promedio
- Detección y eliminación de outliers
- Pesos por distancia
- Robust scaling

4. Comparación con otros algoritmos:

- Regresión lineal: Menos sensible a outliers
- Random Forest: Más robusto
- SVM: Más robusto con kernels apropiados

2.3.6. Selección de Variables

1. Para elegir variables importantes:

- Análisis de correlación
- Feature importance con otros algoritmos
- Recursive feature elimination
- Lasso regression

2. Técnicas que funcionan bien:

- Filtros basados en correlación
- Wrapper methods
- Embedded methods

3. Es importante porque:

- Reduce curse of dimensionality
- Mejora interpretabilidad
- Reduce ruido en los datos
- Acelera computación

2.4. Comparación con Otros Métodos

1. Comparación con regresión lineal:
 - KNN: No paramétrico, captura relaciones no lineales
 - Lineal: Paramétrico, asume relación lineal
 - KNN: Más flexible pero más propenso a overfitting
 - Lineal: Más interpretable y eficiente
2. Comparación con regresión polinomial:
 - KNN: No requiere especificar grado del polinomio
 - Polinomial: Más eficiente computacionalmente
 - KNN: Mejor para relaciones complejas
 - Polinomial: Mejor para relaciones suaves
3. Preferencias:
 - KNN: Datos complejos, pocas variables
 - Lineal: Relaciones lineales, interpretabilidad
 - Polinomial: Relaciones suaves, eficiencia

2.5. Análisis de Datos Manual

1. Datos normalizados (min-max scaling):

Matemática Norm	Física Norm	Química Norm	Aprobó
0.74	0.57	0.70	Sí
0.26	0.11	0.10	No
0.93	1.00	1.00	Sí
0.11	0.36	0.35	No
0.85	0.82	0.60	Sí
0.37	0.29	0.00	No
1.00	1.00	1.00	Sí
0.19	0.21	0.20	No
0.74	0.64	0.50	Sí
0.00	0.00	0.00	No

2. Punto a clasificar normalizado: (0.56, 0.48, 0.50)

Distancias euclidianas:

- A: $\sqrt{(0,56 - 0,74)^2 + (0,48 - 0,57)^2 + (0,50 - 0,70)^2} = 0,23$
- B: $\sqrt{(0,56 - 0,26)^2 + (0,48 - 0,11)^2 + (0,50 - 0,10)^2} = 0,47$
- C: $\sqrt{(0,56 - 0,93)^2 + (0,48 - 1,00)^2 + (0,50 - 1,00)^2} = 0,67$
- D: $\sqrt{(0,56 - 0,11)^2 + (0,48 - 0,36)^2 + (0,50 - 0,35)^2} = 0,47$
- E: $\sqrt{(0,56 - 0,85)^2 + (0,48 - 0,82)^2 + (0,50 - 0,60)^2} = 0,35$
- F: $\sqrt{(0,56 - 0,37)^2 + (0,48 - 0,29)^2 + (0,50 - 0,00)^2} = 0,52$
- G: $\sqrt{(0,56 - 1,00)^2 + (0,48 - 1,00)^2 + (0,50 - 1,00)^2} = 0,75$
- H: $\sqrt{(0,56 - 0,19)^2 + (0,48 - 0,21)^2 + (0,50 - 0,20)^2} = 0,40$
- I: $\sqrt{(0,56 - 0,74)^2 + (0,48 - 0,64)^2 + (0,50 - 0,50)^2} = 0,18$
- J: $\sqrt{(0,56 - 0,00)^2 + (0,48 - 0,00)^2 + (0,50 - 0,00)^2} = 0,78$

3. Los 3 vecinos más cercanos son:

- a) I (0.74, 0.64, 0.50) - Sí (distancia: 0.18)
- b) A (0.74, 0.57, 0.70) - Sí (distancia: 0.23)
- c) E (0.85, 0.82, 0.60) - Sí (distancia: 0.35)

4. Predicción: Sí (3 votos a favor)

5. Con k=5, los vecinos adicionales serían:

- a) H (0.19, 0.21, 0.20) - No (distancia: 0.40)
- b) D (0.11, 0.36, 0.35) - No (distancia: 0.47)

Predicción: Sí (3 votos a favor vs 2 en contra)